

Below is a production-grade **High Level Design (HLD)** for the **Granthan Auth Service**, assuming:

- Email + Password authentication only
 - No OAuth (Google/GitHub/etc.)
 - No Roles & Permissions
 - JWT + Refresh Tokens
 - Microservice Architecture
 - PostgreSQL + Redis + Kafka
 - Notification Service handles emails
 - API Gateway in front of all services
-

Auth Service HLD

1. Overview

Purpose

The Auth Service is responsible for **authentication and user identity management** across the Granthan platform.

It provides secure user registration, login, session management, token generation, password management, and email verification while remaining independent of business logic such as conversations, files, memory, or graph processing.

Responsibilities

- User Registration
- User Login
- User Logout
- JWT Access Token Generation
- Refresh Token Rotation
- Session Management
- Password Hashing
- Password Reset
- Email Verification
- User Profile
- Authentication Audit Logs
- Publish Authentication Events

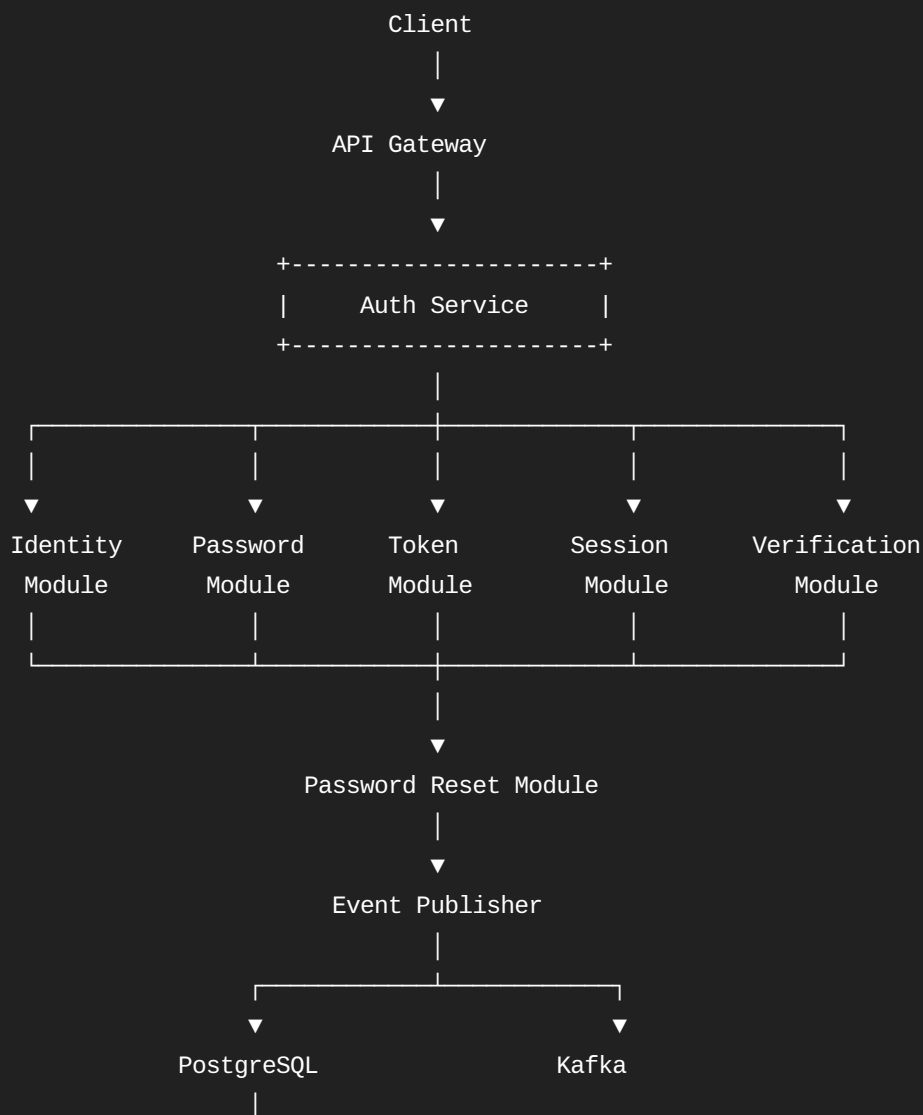
Out of Scope

The Auth Service will **NOT** manage:

- Conversations
- Memories
- Graphs
- Files
- Notifications
- LLMs
- Search
- Analytics

These belong to their own services.

2. High Level Architecture



3. Internal Modules

Identity Module

Responsibility

Maintains user accounts.

Handles

- Register User
- Find User
- Update Profile
- Update Email
- Fetch Current User

Database

users

Why?

Everything starts with the user identity.

Every authentication request first loads the user.

Password Module

Responsibility

Secure password handling.

Handles

- Password Validation
- Password Hashing
- Password Verification
- Password Change

Uses

Argon2id

Why?

Authentication logic should never know how passwords are hashed.

If tomorrow hashing changes, only this module changes.

Token Module

Responsibility

JWT generation and validation.

Handles

- Generate Access Token
- Validate Token
- Parse Claims
- Verify Signature

Token

JWT

Expiration: 15 Minutes

Why?

Keeps JWT implementation isolated from business logic.

Refresh Token Module

Responsibility

Long-lived authentication.

Handles

- Generate Refresh Token
- Rotate Refresh Token
- Validate Refresh Token
- Revoke Refresh Token

Why?

Access tokens expire quickly.

Refresh tokens provide secure long-term login.

Session Module

Responsibility

Track active devices.

Each login creates one session.

Stores

- Device
- Browser
- Operating System
- IP Address
- Login Time
- Last Seen

Supports

- Logout Current Device
 - Logout All Devices
 - List Active Sessions
-

Verification Module

Responsibility

Email ownership verification.

Handles

- Generate Verification Code
- Validate Code
- Resend Verification
- Expiration

Publishes event

EmailVerificationRequested

Notification Service sends email.

Password Reset Module

Responsibility

Recover forgotten password.

Handles

- Generate Reset Token
- Validate Token
- Reset Password
- Revoke Sessions

Publishes

PasswordResetRequested

Audit Module

Responsibility

Maintain authentication history.

Stores

- Login
- Logout
- Failed Login
- Password Change
- Password Reset
- Email Verification

Useful for

- Security
 - Analytics
 - Debugging
-

Event Publisher

Responsibility

Inform other services.

Publishes

UserRegistered

UserLoggedIn

UserLoggedOut

PasswordChanged

PasswordReset

EmailVerified

Consumers

Notification Service

Analytics Service

Search Service (optional)

4. Database Design

PostgreSQL

users

id

email

password_hash

full_name

avatar_url

is_email_verified

status

created_at

updated_at

last_login

refresh_tokens

id

user_id

token_hash

expires_at

revoked

created_at

sessions

id

user_id

refresh_token_id

device

browser

os

ip_address

last_seen

expires_at

email_verifications

```
id
user_id
verification_code
expires_at
used
created_at
```

password_resets

```
id
user_id
reset_token
expires_at
used
created_at
```

audit_logs

```
id
user_id
action
ip_address
user_agent
metadata
created_at
```

Redis

Stores temporary information.

Rate Limiting

OTP Cache

Verification Code Cache

Login Attempts

JWT Blacklist (optional)

Short-lived Session Cache

Redis is used because all of this data is temporary and benefits from fast in-memory access with automatic expiration.

5. API Design

Public APIs

POST /auth/register

POST /auth/login

POST /auth/logout

POST /auth/refresh

POST /auth/verify-email

POST /auth/resend-verification

POST /auth/forgot-password

POST /auth/reset-password

Protected APIs

GET /auth/me

PATCH /auth/profile

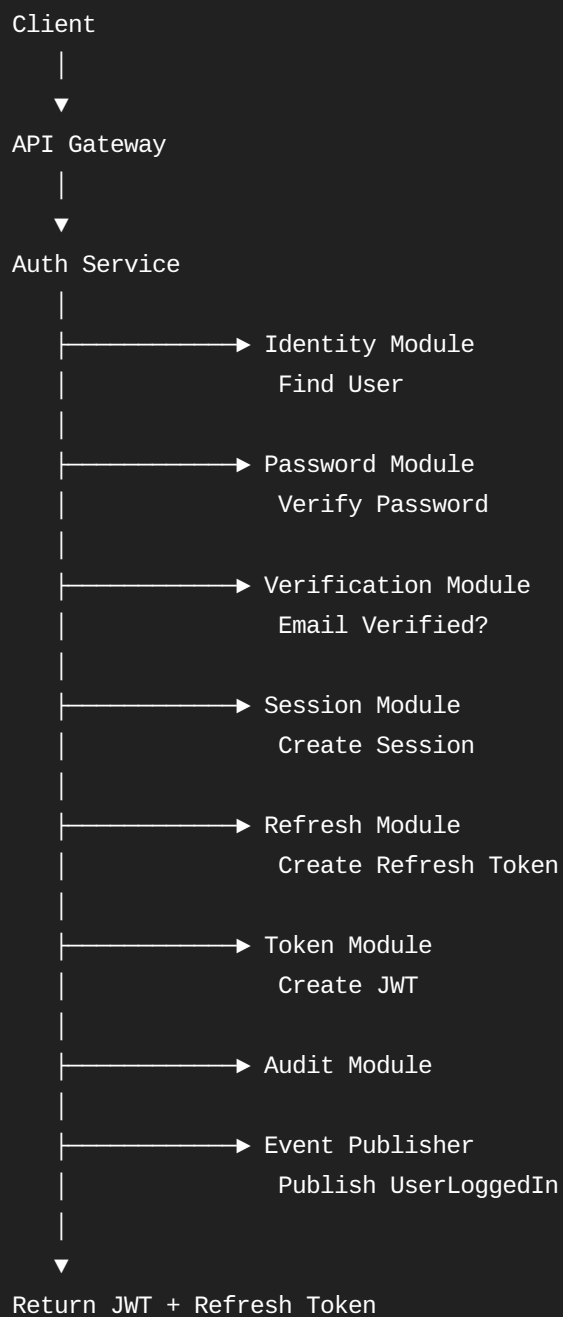
POST /auth/change-password

GET /auth/sessions

DELETE /auth/sessions/{id}

DELETE /auth/sessions

6. Authentication Flow



7. Registration Flow

Client

↓

Register

↓

Validate Request

↓

Check Existing Email

↓

Hash Password

↓

Create User

↓

Generate Verification Code

↓

Store Verification Record

↓

Publish EmailVerificationRequested

↓

Notification Service Sends Email

↓

Registration Complete

8. Password Reset Flow

Forgot Password

↓

Generate Secure Reset Token

↓

Store Token

↓

Publish PasswordResetRequested

↓

Notification Service Sends Email

↓

User Opens Link

↓

Validate Token

↓

Hash New Password

↓

Update Password

↓

Revoke Refresh Tokens

↓

Delete Active Sessions

9. JWT Structure

JSON

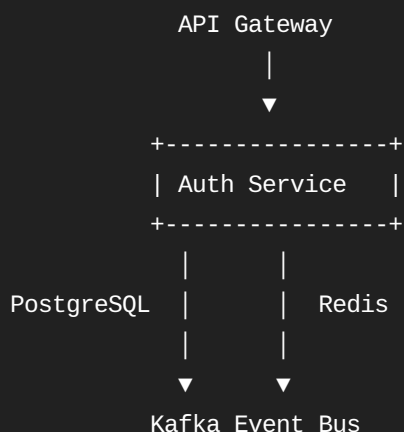
```
{  
  "sub": "user_id",
```

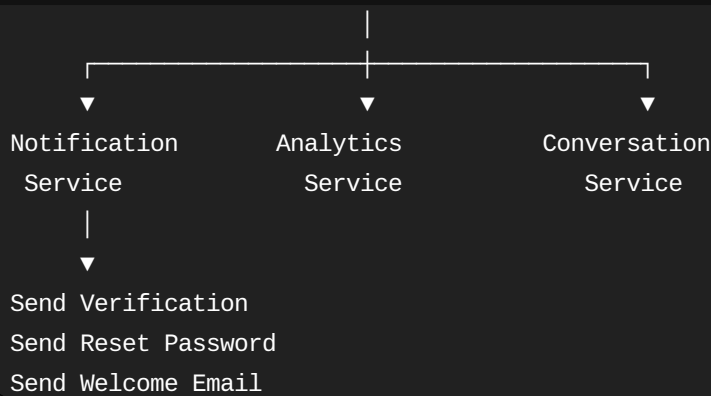
```
"email": "alice@example.com",
"email_verified": true,
"iat": 1721000000,
"exp": 1721000900
}
```

10. Security Measures

Feature	Implementation
Password Hashing	Argon2id
Password Comparison	Constant-time comparison
Access Token	JWT (15 minutes)
Refresh Token	Random 256-bit token, stored as a hash
HTTPS	Mandatory
Email Verification	Required before login (optional policy)
Password Reset	Single-use, time-limited tokens
Rate Limiting	Redis-based per IP and per email
Session Revocation	Supported via refresh token invalidation
Audit Logging	Every authentication event recorded

11. Communication with Other Services





12. Why This Design?

- **Single Responsibility:** The service focuses solely on authentication and identity.
- **Modular Design:** Passwords, tokens, sessions, and verification are isolated, making them easier to test and evolve.
- **Scalability:** Stateless JWT authentication allows multiple Auth Service instances behind a load balancer, while PostgreSQL, Redis, and Kafka provide scalable persistence, caching, and event-driven integration.
- **Security:** Argon2id hashing, hashed refresh tokens, short-lived access tokens, audit logging, and rate limiting follow modern security best practices.
- **Loose Coupling:** By publishing events to Kafka, the Auth Service doesn't need to know about email delivery, analytics, or other downstream concerns.

This HLD provides a solid foundation for a production-ready authentication service that is secure, maintainable, and fits naturally into the overall Granthan microservices architecture.